

**Московский государственный технический
университет им. Н.Э. Баумана**

Факультет «Информатика и системы управления»
Кафедра ИУ5 «Системы обработки информации и управления»

Курс «Парадигмы и конструкции языков программирования»
Отчет по домашнему заданию

Выполнил:
Студент группы ИУ5-31Б
Ростинин Марк

Проверил:
Гапанюк Ю.Е.

2025 г.

1. Введение

Цель данного проекта — создать минимальный рабочий прототип **Visual Transformation Graph** на языке **Rust**, демонстрирующий принципы **dataflow-парадигмы, модульности и полиморфизма**. Проект предназначен для обработки изображений, где каждый узел графа выполняет отдельную трансформацию. Такой подход позволяет создавать **расширяемые конвейеры обработки данных**, аналогичные тем, что используются в современных технологиях дипфейков.

В рамках данного проекта были реализованы следующие функциональные блоки:

- загрузка изображения (LoadImage),
- преобразование в оттенки серого (Grayscale),
- сохранение изображения (SaveImage).

2. Архитектура:

Проект реализован с использованием **модульной структуры Rust**:

visual_transformation_graph/

├── Cargo.toml

└── src/

 ├── main.rs

 ├── graph.rs

 ├── frame.rs

 └── nodes/

 ├── mod.rs

 ├── load.rs

 ├── grayscale.rs

 └── save.rs

Компоненты:

1. Frame (frame.rs)

Структура, инкапсулирующая изображение в виде Mat из OpenCV. Используется паттерн **immutable data**, что повышает безопасность владения памятью.

2. Graph (graph.rs)

- Интерфейс Node описывает узел графа: метод process принимает Frame и возвращает новый Frame.
- Graph хранит вектор узлов Vec<Box<dyn Node>> и запускает их последовательно.
- Здесь применяется **динамический полиморфизм** через Box<dyn Node>, что позволяет хранить узлы разных типов в одном контейнере.

3. Nodes (nodes/*.rs)

Каждый узел реализует свой функционал:

- LoadImage — загрузка изображения с диска.
- Grayscale — преобразование в оттенки серого через OpenCV.
- SaveImage — сохранение изображения на диск.

4. Main (main.rs)

Сборка графа и запуск пайплайна:

```
let graph = Graph::new()
    .add(LoadImage { path: "input.jpg".to_string() })
    .add(Grayscale)
    .add(SaveImage { path: "output.jpg".to_string() });

graph.run(Frame::new(opencv::core::Mat::default()))?;
```

3. Используемые парадигмы и конструкции

Элемент	Парадигма / Конструкция	Комментарий
trait Node	Абстракция / интерфейс	Позволяет разным узлам иметь одинаковый интерфейс
Box<dyn Node>	Динамический полиморфизм	

		Позволяет хранить разные типы узлов в одном векторе
Graph.run	Dataflow / функциональная композиция	Frame проходит через все узлы последовательно
Frame с Mat	Иммутабельность и владение	Rust гарантирует безопасное управление памятью
Модули (mod)	Модульность	Отделение логики загрузки, обработки и сохранения
Result<T>	Обработка ошибок	Используется anyhow::Result для безопасной работы с OpenCV

4. Комментарий к коду:

4.1 frame.rs

```
#[derive(Clone)]
pub struct Frame {
    pub mat: Mat,
}
```

Хранит изображение.

Clone позволяет безопасно передавать данные между узлами без нарушения владения

4.2 graph.rs

```
pub trait Node {
    fn process(&self, input: Frame) -> Result<Frame>;
}
```

Определяет **контракт узла**.

Любой новый узел, например FaceDetect, может быть добавлен без изменения Graph.

```
pub struct Graph {
    nodes: Vec<Box<dyn Node>>,
}
```

Хранит узлы в динамическом векторе, обеспечивая гибкость.

```
pub fn run(&self, mut frame: Frame) -> Result<Frame> {
    for node in &self.nodes {
        frame = node.process(frame)?;
    }
    Ok(frame)
}
```

Последовательно выполняет все узлы, демонстрируя dataflow-подход.

4.3 Узлы

LoadImage:

```
let mat = imgcodecs::imread(&self.path, imgcodecs::IMREAD_COLOR)?;
```

Загружает изображение через OpenCV.

Ошибки обрабатываются через Result.

Grayscale:

```
imgproc::cvt_color(&input.mat, &mut gray, imgproc::COLOR_BGR2GRAY, 0)?;
```

Применяет трансформацию цвета.

Демонстрирует **чистую функцию** на узел.

SaveImage:

```
imgcodecs::imwrite(&self.path, &input.mat, &opencv::types::VectorOfI32::new())?;
```

Сохраняет результат.

Можно легко заменить на другой формат или узел трансформации.

5. Расширяемость проекта

- Добавление новых узлов: FaceDetect, Warp, Blend
- Конфигурация графа через JSON
- Подключение ML-моделей через ONNX для генерации дипфейков
- Работа с видеофайлами вместо изображений

Благодаря **динамическому полиморфизму и модульной структуре**, проект легко расширяется без изменения существующих узлов.

6. Заключение

Данный проект:

- демонстрирует **основные парадигмы и конструкции Rust**: модульность, traits, ownership, иммутабельность, динамический полиморфизм, error handling;
- реализует **минимальный пайплайн обработки изображений**, который может быть использован как основа для более сложных проектов, включая дипфейки;

- служит примером **dataflow-подхода** и функциональной композиции узлов в Rust.

Минимальный рабочий прототип укладывается в рамки учебного проекта и позволяет студенту освоить ключевые концепции языка и парадигмы программирования на практике.